

X3 T1. Convert the automaton from task 1 of the previous exercises (automaton that accepts all binary numbers divisible by four.

- a) into a grammar
- b) into a regular expression

Grammar from DFA

$S \rightarrow 1A \mid 0B$

$A \rightarrow 0S \mid 1A$

$B \rightarrow 0B \mid 1A \mid \varepsilon$

RE from DFA

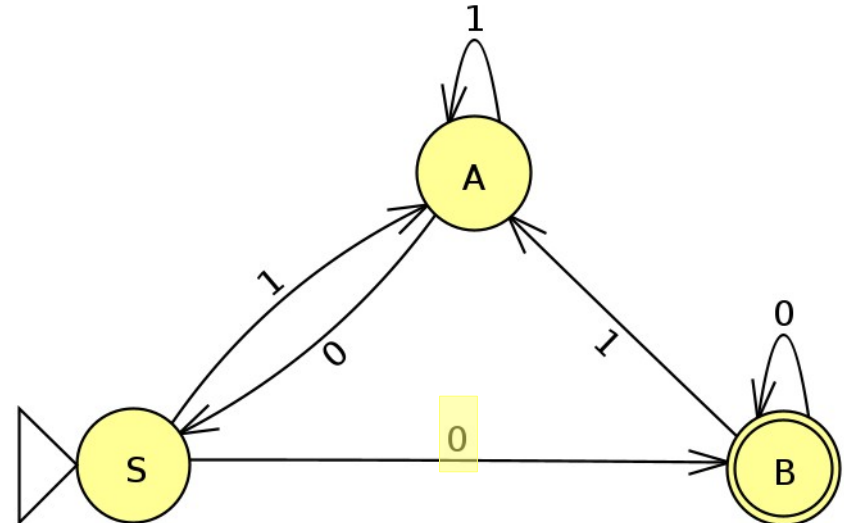
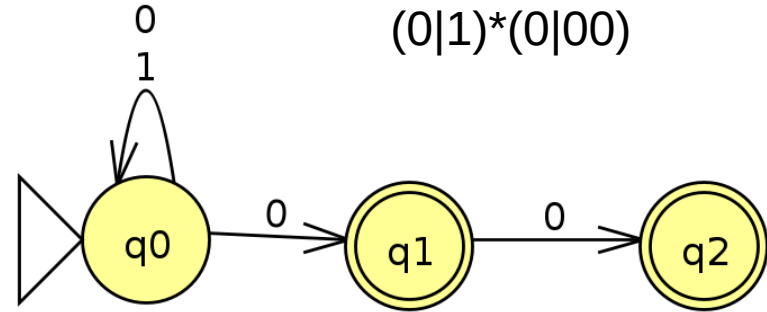
$((11^*0)^*00^*11^*0)^*(11^*0)^*00^*$

There's one mandatory zero (a fixed 0 that must occur in every accepted string), all the other are under * and are optional/ repeatable.

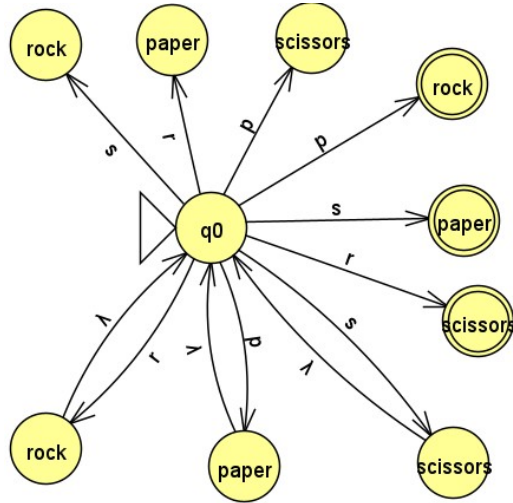
"same" isolated zero within the optional repeatable block

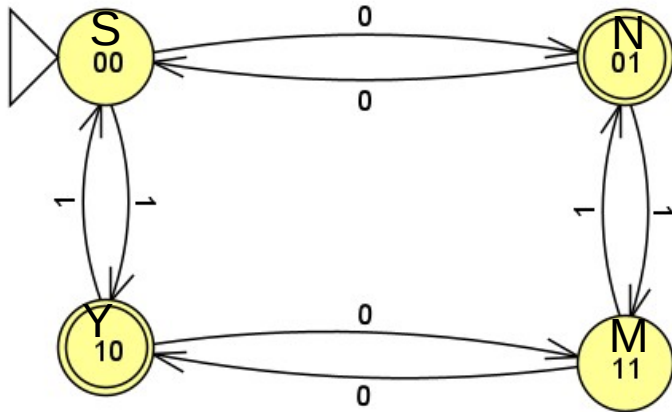
RE from NFA

$(0|1)^*(0|00)$



X3 T2. Convert these lecture examples into grammars. How do these grammars differ from each other?



$$\begin{aligned} S &\rightarrow rR \mid pP \mid sK \mid rO \mid pA \mid rC \\ R &\rightarrow S \\ P &\rightarrow S \\ K &\rightarrow S \\ O &\rightarrow \varepsilon \\ A &\rightarrow \varepsilon \\ C &\rightarrow \varepsilon \end{aligned}$$


$$\begin{aligned} S &\rightarrow 0N \mid 1Y \\ N &\rightarrow 0S \mid 1M \mid \varepsilon \\ Y &\rightarrow 0M \mid 1S \mid \varepsilon \\ M &\rightarrow 0Y \mid 1N \end{aligned}$$

X3 T3. Last week's tasks dealt with Java floating-point constants.

a) Based on which features is the given grammar context-free and not regular?

X2 T5. Familiarize yourself with the grammar notation used in the Java documentation and the representation of floating-point literals (<https://docs.oracle.com/javase/specs/jls/se17/html/jls-3.html#jls-3.10.2> Based on the grammar, create a simplified grammar for DecimalFloatingPointLiteral using the notation used in the lecture.

FloatingPtLiteral $\rightarrow$ DecFloatingPtLiteral

DecFloatingPtLiteral $\rightarrow$ Digits.MaybeDigits MaybeExponentPart MaybeFloatTypeSuffix

| .Digits MaybeExponentPart MaybeFloatTypeSuffix

| Digits ExponentPart MaybeFloatTypeSuffix

| Digits MaybeExponentPart FloatTypeSuffix

Digits $\rightarrow$ Digit | Digit MaybeDigitsAndUnderscores Digit

Digit $\rightarrow$ 0 | NonZero

NonZero $\rightarrow$ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

MaybeDigits $\rightarrow$ Digits | ϵ

MaybeExponentPart $\rightarrow$ ExponentPart | ϵ

MaybeFloatTypeSuffix $\rightarrow$ FloatTypeSuffix | ϵ

ExponentPart $\rightarrow$ ExponentIndicator SignedInteger

ExponentIndicator $\rightarrow$ e | E

SignedInteger $\rightarrow$ MaybeSign Digits

MaybeSign $\rightarrow$ Sign | ϵ

Sign $\rightarrow$ + | -

FloatTypeSuffix $\rightarrow$ f | F | d | D

MaybeDigitsAndUnderscores $\rightarrow$ DigitsAndUnderscores | ϵ

DigitsAndUnderscores $\rightarrow$ DigitOrUnderscore ManyDigitOrUnderscore

ManyDigitOrUnderscore $\rightarrow$ DigitOrUnderscore ManyDigitOrUnderscore | ϵ

DigitOrUnderscore $\rightarrow$ Digit | _

Not regular because

- multiple nonterminals on right-hand side

- neither left- nor right-linear

Not context-sensitive because

- on left-hand side every rule has single non-terminal (no surrounding context)

$K \rightarrow C$

$C \rightarrow G.IPL|.GPL|GXL|GPF$

$G \rightarrow D | DND$

$D \rightarrow 0 | T$

$T \rightarrow 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

$I \rightarrow G | \epsilon$

$P \rightarrow X | \epsilon$

$L \rightarrow F | \epsilon$

$X \rightarrow EN$

$E \rightarrow e$

$N \rightarrow MG$

$M \rightarrow S | \epsilon$

$S \rightarrow + | -$

$F \rightarrow f | d$

$N \rightarrow A | \epsilon$

$A \rightarrow UO$

$O \rightarrow UO | \epsilon$

$U \rightarrow D | _$

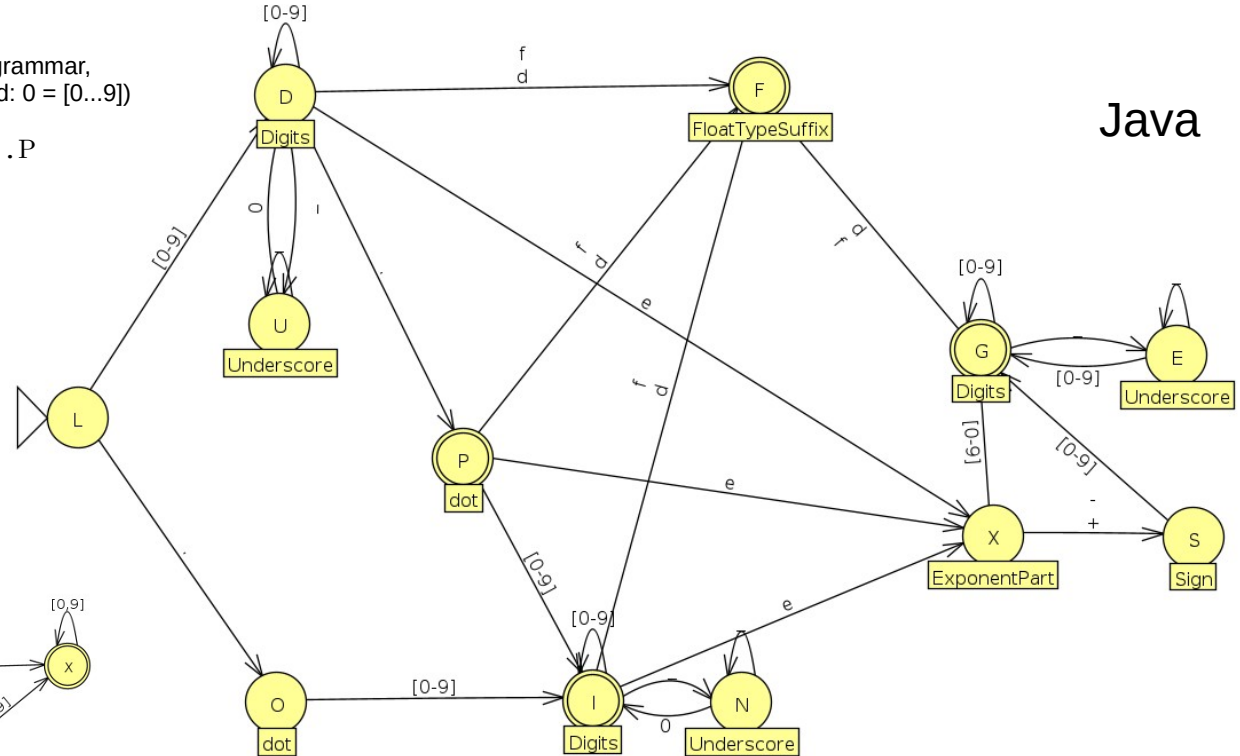
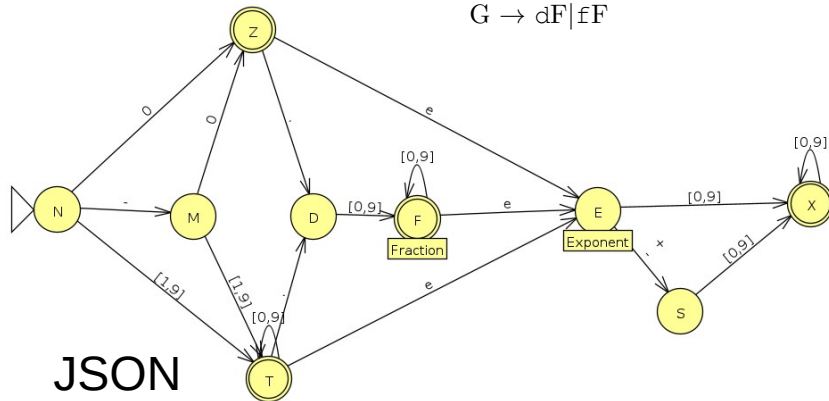
X3 T3. Last week's tasks dealt with Java floating-point constants.

b) Although the given grammar is context-free, the language of floating-point numbers is regular. Demonstrate this by providing a finite automaton that accepts Java floating-point constants. How does a Java floating-point number differ from the JSON number in last week's tasks?

c) Consider why Java floating-point numbers are presented as a context-free grammar in the documentation, if the language of floating-point numbers is regular.

JAVA right-linear regular grammar,
right-linear (here simplified: 0 = [0...9])

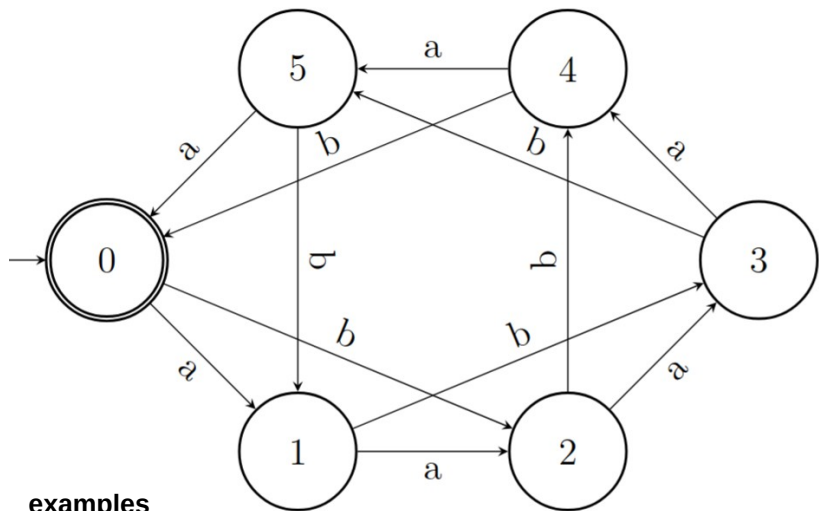
$L \rightarrow 0D \mid .O$
 $D \rightarrow 0D \mid _U \mid fF \mid dF \mid eX \mid .P$
 $U \rightarrow _U \mid 0D$
 $F \rightarrow \epsilon$
 $P \rightarrow \epsilon \mid eX \mid 0I \mid fF \mid dF$
 $O \rightarrow 0I$
 $I \rightarrow \epsilon \mid 0I \mid _N \mid eX$
 $N \rightarrow _N \mid 0I$
 $G \rightarrow \epsilon \mid 0G \mid _E$
 $E \rightarrow _E \mid 0G$
 $X \rightarrow 0G \mid +S \mid -S$
 $S \rightarrow 0G$
 $G \rightarrow dF \mid fF$



X3 T4. Answer based on the image

a) What lengths of words does the automaton accept?

b) What is the formal language it accepts?



examples

accepts ϵ ($|w|=0$)

rejects a,b,aa,ab,ba,bb (all $|w|=1$ or 2)

accepts bbb,aabb,aaaab,aaaaaa ($|w|=3,4,5,6$)

each 'a' worth 1 and each 'b' worth 2,

accepts bbb, aabb, aaaab, aaaaaa or any combination of a's and b's where the weighted count of symbols is multiple of 6

→ for $|w| \geq 3$, it is always possible to choose numbers of a's and b's whose total weight is a multiple of 6, so for $|w| \geq 3$, there exists at least one string of length n that is accepted by the language

→ accepts all $|w| \geq 0$ except $|w|=1$ and $|w|=2$

$$L = \{ w \in \{a,b\}^* \mid |w|_a + 2|w|_b \equiv 0 \pmod{6} \}$$

Strings where the weighted count of symbols is multiple of 6, with each 'a' worth 1 and each 'b' worth 2

total count of symbols $n=x+y$, where x number of a's, y number of b's, $a=1$, $b=2$, the condition $xa+yb \equiv 0 \pmod{6}$ becomes $x+2y \equiv 0 \pmod{6}$

why $|w|=1$ fails? when $n=1$ possible weights $\{1,2\}$
either $x=1$ and $y=0 \rightarrow 1+2=1 \not\equiv 0 \pmod{6}$ ($\rightarrow$ state 1)
or $x=0$ and $y=1 \rightarrow 0+2=2 \not\equiv 0 \pmod{6}$ ($\rightarrow$ state 2)

why $|w|=2$ fails? when $n=2$, possible weights $\{2,3,4\}$
 $x=2, y=0 \rightarrow 2+0=2 \not\equiv 0 \pmod{6}$ ($\rightarrow$ state 2)
 $x=1, y=1 \rightarrow 1+2=3 \not\equiv 0 \pmod{6}$ ($\rightarrow$ state 3)
 $x=0, y=2 \rightarrow 0+4=4 \not\equiv 0 \pmod{6}$ ($\rightarrow$ state 4)

It's always possible to find x and y that satisfies the condition when $n \geq 3$

When $n=3$, possible weights $\{3,4,5,6\}$ (includes 6, so length 3 works)

$n=4$, possible weights $\{4,5,6,7,8\}$

$n=5$, possible weights $\{5,6,7,8,9,10\}$

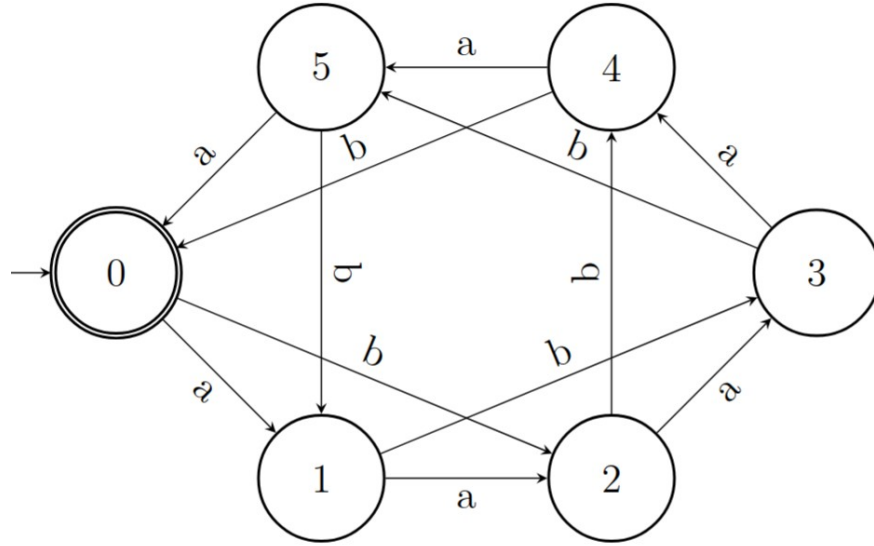
$n=6$, possible weights $\{6,7,\dots,12\}$ (multiple of 6)

$n=7$, possible weights $\{7,8,\dots,12,13,14\}$

Weight span is $[n,2n]$. For $n \geq 3$, $[n,2n]$ always contains multiple of 6. For $n=1$ or $n=2$ it doesn't.

X3 T5. Answer based on the image

a) Is the automaton in the image already minimal or could it still be minimized? Justify your answer briefly.



$$L = \{ w \in \{a,b\}^* \mid |w|_a + 2|w|_b \equiv 0 \pmod{6} \}$$

Strings where the weighted count of symbols is multiple of 6, with each 'a' worth 1 and each 'b' worth 2

Using partition method

$$P_0 = \{q_0\}, \{q_1, q_2, q_3, q_4, q_5\}$$

final states, non-final states

$$P_1 = \{q_0\}, \{q_1, q_2, q_3\}, \{q_4, q_5\}$$

Split because q_4, q_5 go to $\{q_0\}$ group on some input

$$P_2 = \{q_0\}, \{q_1\}, \{q_2, q_3\}, \{q_4, q_5\}$$

Split q_1 alone because only q_2, q_3 go to $\{q_4, q_5\}$ group

$$P_3 = \{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_4, q_5\}$$

Split q_2, q_3 because q_3 goes to $\{q_4, q_5\}$ on both inputs, q_2 goes to q_3 and $\{q_4, q_5\}$

$$P_3 = \{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_4\}, \{q_5\}$$

Split q_4, q_5 , because different groups, $q_4 \rightarrow q_0$ and q_1 , and $q_5 \rightarrow q_0$ and q_1

All states are separate, no states can merge,
DFA was already minimal

X3 T5. Answer based on the image b) If state 3 in the image is changed from rejecting to accepting, what answers would the new automaton give to the questions (a) and (b) in task T4?

→ **What lengths of words does the automaton accept? What is the formal language it accepts?**

examples

accepts $|w|=0$ (ϵ)

rejects all $|w|=1$ (a, b)

accepts some $|w|=2$ (ab, ba)

accepts some $|w|=3$ (aaa,bbb)

From length 2 onward, it's possible to adjust the mix of a's and b's to make the total weight a multiple of 3

→ **accepts all word lengths except $|w|=1$**

why $|w|=1$ fails?

condition $x+2y \equiv 0 \pmod 3$, length $n = x+y$

For $n=1$ ($|w|=1$) either $x=1$ and $y=0$, so $x+2y = 1$

or $x=0$ and $y=1$, so $x+2y = 2$. Neither is congruent to 0 modulo 3.

Since these are the only possibilities for a one-character string, it is impossible for any length 1 string to satisfy $x+2y \equiv 0 \pmod 3$

Possible weights by length

$|w|=0$: {0} ($\equiv 0 \pmod 3$)

$|w|=1$: {1,2}

a: $x=1, y=0 \rightarrow 1 \not\equiv 0 \pmod 3$

b: $x=0, y=1 \rightarrow 2 \not\equiv 0 \pmod 3$

$|w|=2$: {2,3,4} (three included, $3 \equiv 0 \pmod 3$)

aa: $x=2 \rightarrow 2$ ($\rightarrow$ state 2)

ab or ba: $x=1, y=1 \rightarrow 1+2=3$ ($\rightarrow$ state 3)

bb: $x=0, y=2 \rightarrow 4 \equiv 1$ ($\rightarrow$ state 4)

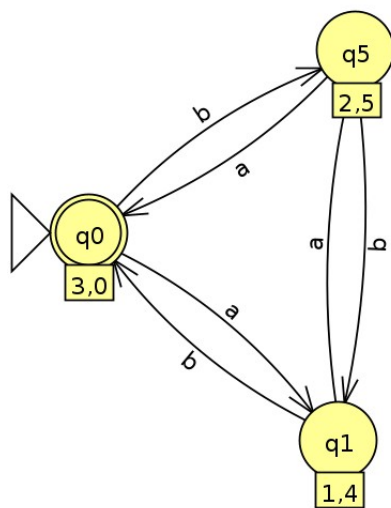
$|w|=3$: {3,4,5,6} ($3 \equiv 0, 6 \equiv 0$)

aaa:3, aab,baa,aba:4, abb,bab:5, bbb:6

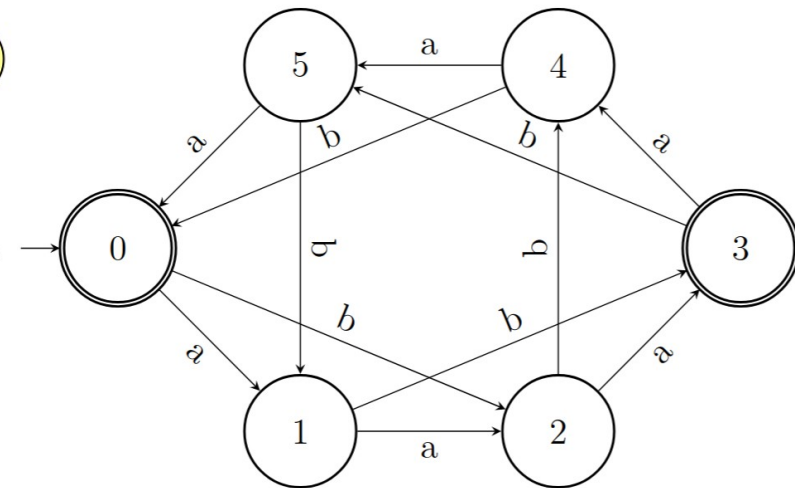
$|w|=4$: {4,5,6,7,8}

Why $|w|=1$ fails: weights {1,2} $\rightarrow$ neither $\equiv 0 \pmod 3$

Why $|w| \geq 2$ works: range $[n, 2n]$ always contains multiple of 3 (span ≥ 2 covers all residues mod 3)



minimized



DFA with two final states

$$L = \{ w \in \{a,b\}^* \mid |w|_a + 2|w|_b \equiv 0 \pmod 3 \}$$

Strings where the weighted count of symbols is a multiple of 3, with each 'a' worth 1 and each 'b' worth 2.

X3 T6. Familiarize yourself with Python's grammar and its NUMBER element. Documentation can be found at https://docs.python.org/3/reference/lexical_analysis.html#numbers. **Write the grammar** for floating-point numbers (NUMBER $\rightarrow$ floatnumber) using the notation from the lectures. Extend the grammar to terminal symbols.

Test the grammar with the numbers 3.14, 1e-10, and 1e. The last one is not accepted; why?

NUMBER $\rightarrow$ Floatnumber
 Floatnumber $\rightarrow$ Digitpart . [Digitpart] [Exponent]
 | . Digitpart [Exponent]
 | Digitpart Exponent
 Digitpart $\rightarrow$ Digit ([_] Digit)*
 Exponent $\rightarrow$ (e | E) [+|-] Digitpart
 Digit $\rightarrow$ 0 | 1 | 2 | ... | 9

S $\rightarrow$ F
 F $\rightarrow$ D . OE | . DE | DX
 O $\rightarrow$ D | ϵ (optional digitpart)
 E $\rightarrow$ X | ϵ (optional exponent)
 X $\rightarrow$ eMD | EMD
 M $\rightarrow$ + | - | ϵ
 D $\rightarrow$ IG
 G $\rightarrow$ UD | ϵ
 U $\rightarrow$ _ | ϵ
 I $\rightarrow$ 0 | 1 | 2 | ... | 9

3.14
 S $\rightarrow$ F $\rightarrow$ D.OE
 $\rightarrow$ D.O ϵ (E $\rightarrow$ ϵ)
 $\rightarrow$ D.O
 $\rightarrow$ IG.O (D $\rightarrow$ IG)
 $\rightarrow$ 3G.O (I $\rightarrow$ 3)
 $\rightarrow$ 3 ϵ .O (G $\rightarrow$ ϵ)
 $\rightarrow$ 3.O
 $\rightarrow$ 3.D (O $\rightarrow$ D)
 $\rightarrow$ 3.IG (D $\rightarrow$ IG)
 $\rightarrow$ 3.1G (I $\rightarrow$ 1)
 $\rightarrow$ 3.1UD (G $\rightarrow$ UD)
 $\rightarrow$ 3.1 ϵ D (U $\rightarrow$ ϵ)
 $\rightarrow$ 3.1D
 $\rightarrow$ 3.1IG (D $\rightarrow$ IG)
 $\rightarrow$ 3.14G (I $\rightarrow$ 4)
 $\rightarrow$ 3.14 ϵ (G $\rightarrow$ ϵ)
 $\rightarrow$ **3.14**

1e-10
 S $\rightarrow$ F $\rightarrow$ DX
 $\rightarrow$ IGX (D $\rightarrow$ IG)
 $\rightarrow$ 1GX (I $\rightarrow$ 1)
 $\rightarrow$ 1 ϵ X (G $\rightarrow$ ϵ)
 $\rightarrow$ 1X
 $\rightarrow$ 1eMD (X $\rightarrow$ eMD)
 $\rightarrow$ 1e-D (M $\rightarrow$ -)
 $\rightarrow$ 1e-IG (D $\rightarrow$ IG)
 $\rightarrow$ 1e-1G (I $\rightarrow$ 1)
 $\rightarrow$ 1e-1UD (G $\rightarrow$ UD)
 $\rightarrow$ 1e-1 ϵ D (U $\rightarrow$ ϵ)
 $\rightarrow$ 1e-1D
 $\rightarrow$ 1e-1IG (D $\rightarrow$ IG)
 $\rightarrow$ 1e-10G (I $\rightarrow$ 0)
 $\rightarrow$ 1e-10 ϵ (G $\rightarrow$ ϵ)
 $\rightarrow$ **1e-10**

1e
 S $\rightarrow$ F $\rightarrow$ DX
 $\rightarrow$ IGX (D $\rightarrow$ IG)
 $\rightarrow$ 1GX (I $\rightarrow$ 1)
 $\rightarrow$ 1 ϵ X (G $\rightarrow$ ϵ)
 $\rightarrow$ 1X
 $\rightarrow$ 1eMD (X $\rightarrow$ eMD)
 $\rightarrow$ 1e ϵ D (M $\rightarrow$ ϵ)
 $\rightarrow$ 1eD
 $\rightarrow$ 1eIG (D $\rightarrow$ IG)
 $\rightarrow$ 1eI (G $\rightarrow$ ϵ)
 $\rightarrow$ nonterminal remain
 unreduced, no valid path
 to terminate 1e. Parser
 expects complete
 number, but 1e lacks
 digits after e to finish the
 exponent

```
num = 1e
→ real-time parsing while
typing

num = 1e3
print(num) # 1000.0

num = '1e333'
print(float(num)) # inf

num = '1e'
print(num) # 1e
print(float(num))

ValueError: could not
convert string to
float: '1e'
→ raises error, rejects
incomplete exponent
```